Milestone 5: Parallel Solving, Suit Isomorphism and the Flop

holdem-cfr walkthrough

Where the time goes

At the end of milestone 4 the river solved in a twentieth of a second, but the turn took thirteen seconds. The reason is structural. A turn spot holds one complete river subtree for each of the 48 river cards that could come, so it does about fifty times the river's work per iteration. A flop spot has another layer: 49 turn cards, each followed by about 48 rivers. That is roughly 2,300 river subtrees. No single trick fixes that, so this milestone stacks three:

1. Do the independent work at the same time, on every core. 2. Do not do the same work twice when two cards are strategically identical. 3. Choose the bet menu per street, so the tree is only as large as it needs to be.

A fourth piece is about honesty rather than speed: the solver states how much memory a tree needs before allocating any of it.

Parallelism without unsafe code

Below a chance node, the 48 river subtrees are independent within one traversal. Each one reads the same reach vectors and returns its own value vector, and none of them touches another's regrets. That is textbook data parallelism, and Rust's *rayon* library expresses it in one line: replace `iter()` with `par_iter()`, and rayon's work-stealing scheduler spreads the items across a thread pool.

The difficulty is that the subtrees all *write*. Every river subtree updates its own regrets and strategy sums, and those live in two big shared arrays per player. Rust's borrow checker will not let several threads hold mutable references into one array at the same time. In most languages you would reach for locks or atomics here, or quietly accept a data race.

The way out comes from a property the tree already has. Nodes are stored in **pre-order** (a parent before its children, each subtree contiguous). Array offsets are handed out in node order. So **every subtree owns one contiguous slice** of each player's arrays, and sibling subtrees own disjoint slices in increasing order. The solver precomputes each subtree's slice (its "span") with one reverse sweep over the nodes. At a chance node it cuts the arrays into per-child pieces with `split_at_mut`:

```
for &child in children {
    let (lo, hi) = ctx.span[child][p];
    let (_, tail) = std::mem::take(&mut rest[p]).split_at_mut(lo - cursor[p]);
    let (mine, tail) = tail.split_at_mut(hi - lo);
    rest[p] = tail;          // what is left for the next sibling
    // `mine` is this child's exclusive slice
}
```

`split_at_mut` is the standard-library primitive that turns one `&mut [T]` into two non-overlapping `&mut [T]`. Each child gets a `View`: an exclusive slice of each player's arrays plus the global offset the slice starts at. The children then run with `par_iter`. The compiler has *proved* there is no data race, because each thread holds a mutable reference to memory that no other thread can reach. The whole parallel solver is safe Rust.

One bug along the way is worth telling. The first version took the parent's slices with `std::mem::take`, which moves the reference out and leaves an empty slice behind. The chance node worked, but when it returned, the *parent* action node tried to process its next action and found its own slice empty. The fix is to **reborrow** instead of take:

```
let [r0, r1] = &mut view.regrets;
let mut rest: [&mut [f32]; 2] = [&mut **r0, &mut **r1];
```

The reborrow is a temporary `&mut` derived from the parent's. The borrow checker guarantees it ends before the parent uses its slice again, and nothing is moved out. Destructuring `[r0, r1]` matters too. Writing `view.regrets[0]` and `view.regrets[1]` would not compile, because array indexing does not tell the borrow checker that the two elements are disjoint, while a pattern does.

Determinism. Parallel floating-point code often produces slightly different results from run to run, because addition is not associative and threads finish in different orders. Here, child value vectors are collected into a `Vec` *in child order*, and only then summed sequentially, in the same order the serial code uses. The result is **bit-for-bit identical whatever the thread count**. The milestone gate enforces this: six iterations of a turn solve in a one-thread pool and a four-thread pool must produce identical regret arrays, compared as raw bits, and identical exploitability. Reproducibility matters for a research tool. When two runs disagree you want to be sure it is your change and not the scheduler.

On the 2-core development VM, a 100-iteration turn solve goes from 9.2 seconds on one thread to 4.8 seconds on two, a 1.9× speedup. The work splits into 48 roughly equal pieces, so it should keep scaling across all the cores of an M-series laptop.

Suit isomorphism

Suits have no rank in poker, so any relabelling of the four suits gives an equivalent game. Take the turn board Q♠ J♠ 7♣ 4♣, which has no hearts or diamonds, against ranges written as hand classes (AKs, QQ and so on), which treat every suit alike. Swapping hearts with diamonds leaves the board unchanged and both ranges unchanged. So a heart river and its diamond twin, say 9♥ and 9♦, lead to *mirror-image* subgames. Hand h's value after 9♦ equals the value of its mirror σ(h) after 9♥. Solving both is wasted work.

The solver handles this in three steps.

Finding the symmetries. `Spot::new` tries all 24 suit permutations and keeps those that map the starting board onto itself and map every hand in each range to a hand of the same weight. This check is what makes the optimization *exact*. A range containing a lone A♥K♥ breaks the heart–diamond symmetry, so it is not used. The surviving permutations form a group. For each one, the spot precomputes the hand permutation `inverse[g][p][h]`: the index of the hand that symmetry g maps onto hand h.

Merging chance cards. At each chance node, the tree builder keeps only the symmetries that fix the *current* board. A turn card can break a symmetry the flop had. Two cards are equivalent if one of these symmetries maps one onto the other. Each class keeps one canonical card, the lowest image, which gets a real subtree. Every other card in the class is recorded as a *member*, together with the symmetry that produces it.

Using the canonical subtree for its whole class. When values come back up through the chance node, each member contributes the canonical values with hands relabelled:

```
for &(_, g) in class {
    let inv = &spot.inverse[g][trav];
    for (h, o) in out.iter_mut().enumerate() {
        *o += p * v[inv[h] as usize];
    }
}
```

The regrets stored in a canonical subtree serve the whole class. By symmetry, a member's regrets would be the same numbers with hands permuted, so they never need storing.

The savings depend on the board. A rainbow board uses all four suits, and on a rainbow turn there is nothing to merge. A two-tone turn has two missing suits, which merge, so 48 rivers become 35: about 27% less memory and time. On a monotone flop, three suits are interchangeable, so 49 turn cards collapse to 23 classes, and the saving compounds at the river.

The gates. The tests check the exact class counts: 48 on a rainbow turn, 35 on a two-tone turn, 23 on a monotone flop. They check that a lone A♥K♥ disables merging. Most importantly, they check that an isomorphic solve and a full solve of the same two-tone turn agree on both best-response values and both EVs to within 0.001 chips after one iteration and after forty. Following the habit from milestone 4, I confirmed the test can fail. Replacing the hand relabelling with the identity, a plausible bug, makes the first-iteration best response differ by 0.008 chips, and the test catches it.

Per-street bet menus and a memory budget

Commercial solvers let you choose sizes street by street, and the flop shows why. Every size on the flop multiplies the size of everything below it. The bet configuration now holds sizes per street and per player. The CLI exposes them as `--flop-bets`, `--turn-bets`, `--river-bets` and the matching raise flags. A typical lean flop tree is one size on the flop, one on the turn and one or two on the river.

Memory is easy to predict exactly. Each action at each node stores one regret and one strategy sum, as 4-byte floats, for every hand of the acting player. `required_memory` adds this up from the public tree *before* the solver allocates anything. The CLI refuses trees above `--max-memory-mb` (default 4 GB) with a message saying what to change. Crashing the laptop's swap thirty minutes into a run is not a good user experience.

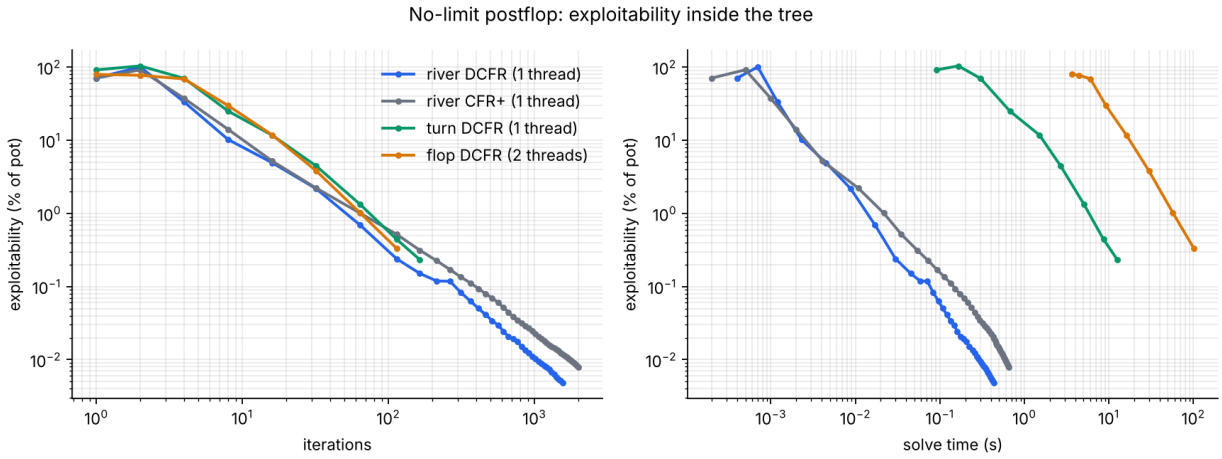
The flop

Putting it all together, here is a full flop solve on the two-tone board Q♠ J♠ 4♣. OOP has 267 combos and IP has 331, with 4× pot behind. The bet menu is lean: 33% of pot on the flop, 66% on the turn, 75% on the river, pot-sized raises, at most a bet and one raise per street, and all-in always available.

The tree has 293,390 public nodes, 106,980 of them decision nodes. That is 588 MB of regrets and strategy sums after isomorphism merged the heart and diamond turn cards. On the 2-core VM, each iteration takes about 0.85 seconds. Exploitability falls from 80% of pot to 3.8% after 32 iterations, 1.0% after 64, and **0.33% of pot after 114 iterations and 101 seconds** of solving. Commercial flop solves are usually stopped somewhere between 0.3% and 0.5% of pot, so this is a usable flop solve on a laptop-class machine, with exploitability measured exactly inside the tree.

On a MacBook Air with 8 threads, the same solve reaches 0.33% of pot in **31 seconds**, 3.2× faster than the VM. It also printed *exactly the same exploitability at every checkpoint*, to the last digit: 79.521%, 77.306%, 68.837% and so on. That holds across a different CPU architecture (ARM instead of x86), a different thread count (8 instead of 2) and a different machine. This is the payoff of fixed-order summation.

Rust also never silently fuses multiplies and adds into FMA instructions, so IEEE 754 arithmetic produces identical bits on both chips.



Postflop convergence

The strategy deserves a caveat that applies to every solver. OOP shoves all-in for 4× pot with 23% of its aces. That does not mean real players should overbet-shove aces on the flop. The tree offers only two bet sizes, 33% and all-in, so a hand that wants to bet big has nowhere else to go. Equilibria are equilibria *of the tree you gave the solver*. That is exactly the limitation the README states, and the reason milestone 6 needs action translation for sizes off the menu.

What this milestone does not do yet

Three directions remain, in rough order of payoff.

Storage precision. Regrets and strategy sums are 32-bit floats. Commercial solvers compress them to 16-bit integers with a per-node scale, which halves memory at a small cost in precision.

Better parallelism on the river. On river-only spots there is no chance node to split on, so the river still runs on one core. It is already a twentieth of a second, so this matters little.

Fewer allocations. The hot loop allocates small vectors per node per traversal. A preallocated scratch arena would remove that, which is mostly a constant-factor win.

None of these change any answer, and all of them sit behind tests that already pin the answers down.

Reading

Johanson et al. (2012), *Efficient Nash Equilibrium Approximation through Monte Carlo CFR*, on public chance sampling: another way to cut the chance branching

factor, by sampling public cards instead of enumerating them. Waugh (2013), *A Fast and Optimal Hand Isomorphism Algorithm*, for the indexing problem that isomorphism raises once boards and hands must be canonicalized together. That is what milestone 6's bucketing needs.